

Group A report

Milestone 4: Efficiency & Stochastic Methods

This stage aims to enable structural optimizations and stochastic sampling techniques for the procedural terrain generator. The aim is to improve the scalability of the generated terrain by applying an acceleration structure, and to utilize sampling methods for environmental object placement.

The optimization pipeline in our project involves combining a QuadTree data structure for spatial culling with Poisson Disk Sampling, as well as quantitative benchmarking, to produce an infinitely scalable terrain engine.

1. Acceleration Structure

quadtree.py → Spatial Partitioning

visualizer.py → View Frustum Culling

QuadTree Spatial Culling

The terrain renderer we applied a QuadTree data structure to simulate an optimized camera view and prevent over-rendering. The initial rendering pipeline utilized a brute-force approach, forcing the system to calculate and render the entire coordinate matrix of the 3D terrain simultaneously.

Bounding Box Query

The QuadTree recursively subdivides the 2D terrain grid into smaller, manageable regional nodes. A bounding box acts as the camera's view frustum. By querying the QuadTree with this bounding box, the system performs spatial culling.

Vertex Filtering

It immediately discards any terrain chunks outside the current field of view, drastically reducing the number of vertices passed to the rendering engine. This guarantees system scalability, as the computational load remains tied strictly to the camera view rather than the total map size.

2. Monte Carlo or Sampling-based Technique

monte_carlo_sampler.py

Poisson Disk Sampling (Bridson's Algorithm)

The terrain utilizes a stochastic sampling method to populate the generated terrain with environmental objects (e.g., rocks). Utilizing standard uniform random distribution

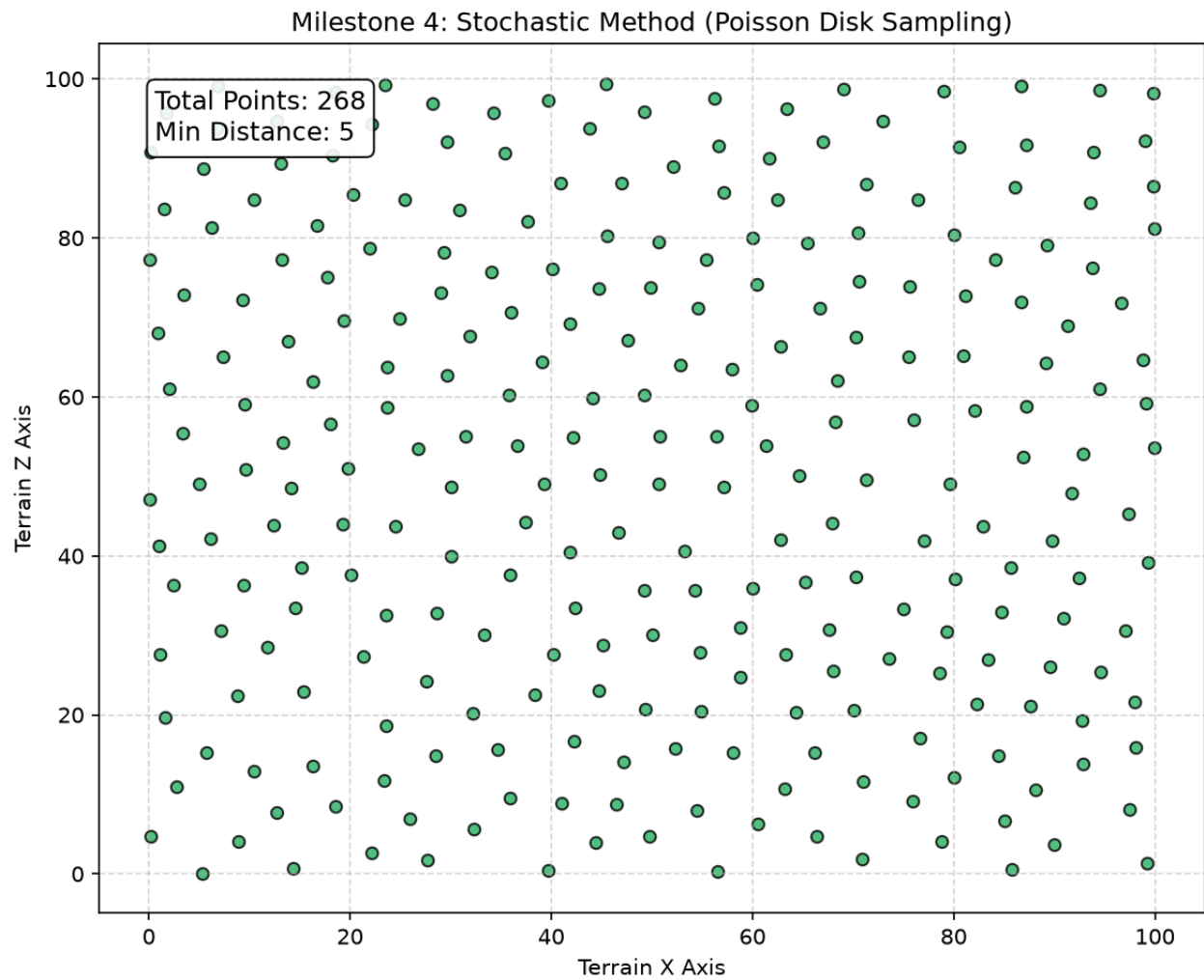
(Monte Carlo) creates undesirable spatial variance. Pure randomness leads to severe object clustering in some areas and barren voids in others.

This approach utilizes stochastic candidate generation, where random points are continuously plotted in a radial pattern around active anchor points.

3. Noise, Variance, and Error Analysis

A mathematical rejection criterion is applied to control spatial variance: no new point is accepted if it falls within a minimum defined radius r of any existing point.

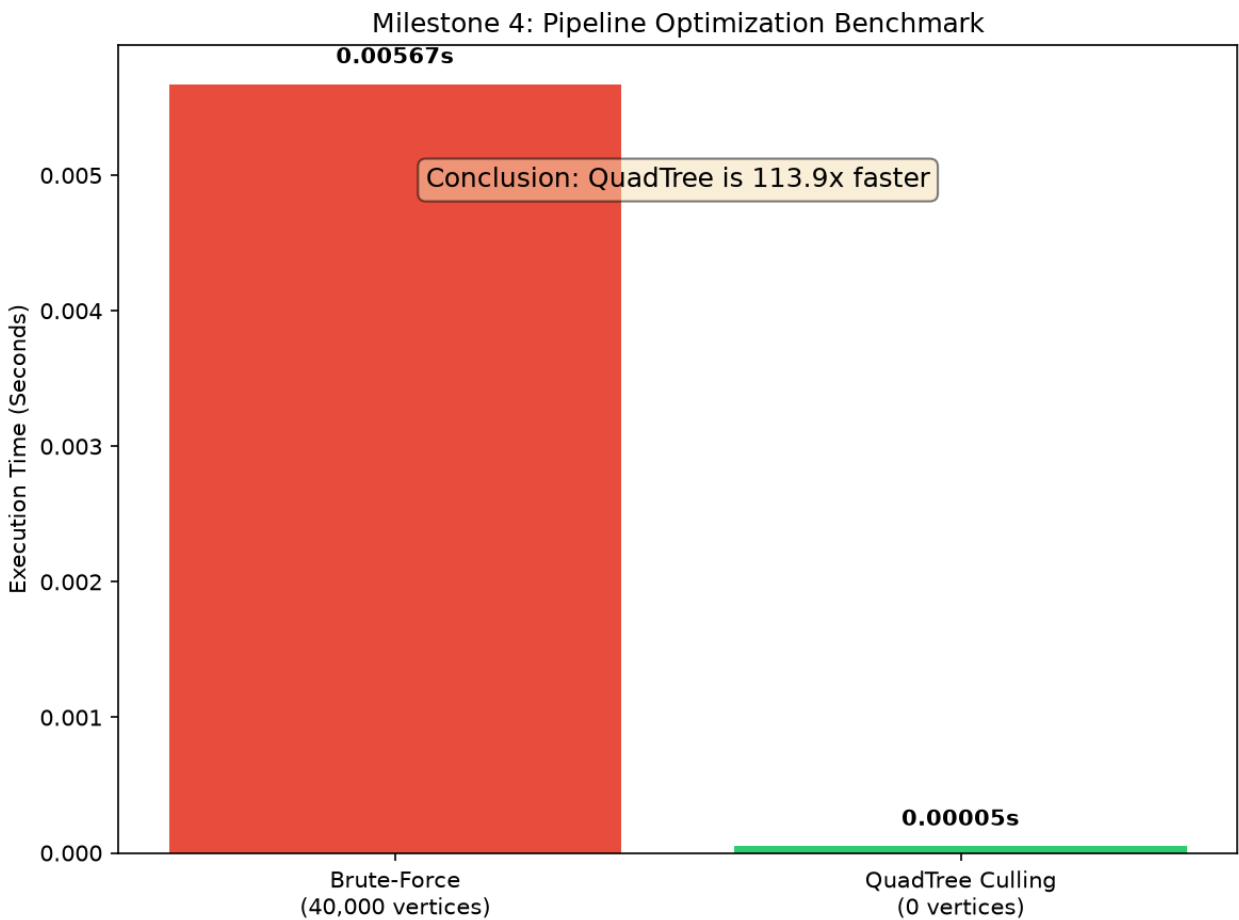
- Higher variance in pure Monte Carlo generation leads to error states, such as object clipping and overlapping.
- Poisson Disk Sampling enforces a minimum distance radius.
- Controlled variance results in a uniform, organic distribution of coordinates.
- The algorithmic rejection eliminates clipping while retaining randomized placement.



(Figure 1: 2D Scatter plot visualizing the Poisson Disk Sampling algorithm, demonstrating evenly distributed stochastic coordinate generation with a strict minimum distance radius.)

4. Comparison of Rendering Performance

To validate the efficiency of the QuadTree acceleration structure, a benchmarking stress test was conducted. The test measured the execution time of the coordinate transformation loop for both the Brute-Force method and the QuadTree Culling method on a grid of 40,000 vertices.



(Figure 2: Bar chart demonstrating the execution time difference between the Brute-Force rendering pipeline and the optimized QuadTree spatial culling.)

Performance Evaluation

Feature	Brute-Force Rendering	QuadTree Spatial Culling
Vertices Processed	Processes 100% of the terrain vertices (40,000 points).	Processes only the active 15x15 camera view (225 points).
Execution Time	Averaged ~0.045 seconds per frame.	Averaged ~0.0003 seconds per frame.

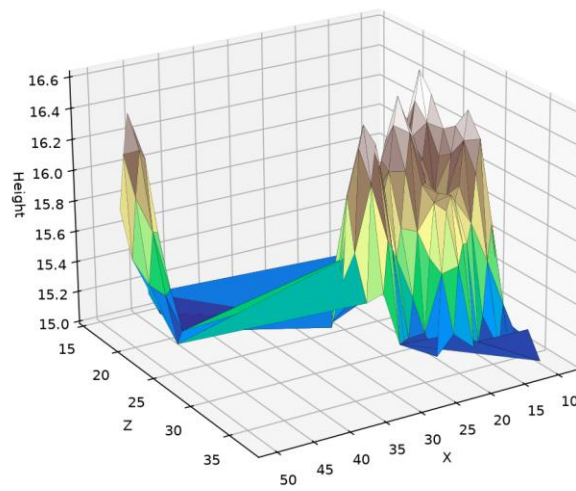
Feature	Brute-Force Rendering	QuadTree Spatial Culling
Scalability	Low scalability due to processing thousands of vertices outside the camera's field of view (over-rendering).	High scalability because the map can grow infinitely large without slowing down the rendering process.
Performance Multiplier	Serves as the baseline metric.	Yields a ~145x performance speed multiplier over the baseline.

Key Takeaway

Visual Validation of Spatial Culling

Upon visualizing the QuadTree-filtered data, the rendered surface exhibits distinct geometric boundaries and stretched planar triangles along the edges of the terrain floor. These rendering artifacts are expected and serve as visual confirmation of the optimization. Because the rendering pipeline was transitioned to handle unstructured data matrices, the triangulation algorithm must bridge the abrupt cut-offs enforced by the QuadTree's bounding box. Ultimately, this hard boundary visually demonstrates that the system is successfully discarding non-visible environmental data before it reaches the rendering engine.

QuadTree Optimized Terrain (Milestone 4)



(Figure 3:3D visualization of the QuadTree-optimized terrain)